AGRITECH

Smart Farming Solutions

Project purpose, feature guide, architecture and implementation overview

Project team	Guide / Academic year
Aryaman Sanskrityayan • 23051090 Ankit Mukherjee • 23051084 Anshika Nath • 2305765 Dev Kumar • 2305292	Ms. Chandani Kumari CSE • 2026–2027

Prepared from the AgriTech repository README and visible project files. Implementation details that depend on runtime configuration are identified as conditional.

1. Executive overview

AgriTech is a browser-based agriculture platform that brings crop decision support and farmer-facing services into one application. The project aims to reduce fragmented access to crop recommendations, fertilizer guidance, yield estimates, plant disease screening, weather and farm information, and community or marketplace resources.

The repository describes a Python Flask application with HTML, CSS, and JavaScript interfaces. Some machine-learning tasks use dedicated model components. Optional integrations such as Gemini, Firebase, email, and external data feeds depend on credentials and configuration. This guide explains the purpose of each feature family, how the user-facing flow works, and how the implementation is organized.

2. Goals and users

• Help a farmer explore recommendations and predictions from a single web application.

• Make plant disease screening accessible from a photo upload workflow.

• Provide information and workflow tools for farm management, markets, community, and public programs.

• Give users a consistent web interface and backend entry point.

The primary user is a farmer. Other roles supported by feature areas include buyers, shopkeepers, administrators, community members, and agricultural service providers. Exact access rules vary by feature and deployment.

3. System overview and request flow

At a high level, the browser collects a user action or input, the Flask server routes the request to application logic or a configured model/service, and the resulting response is rendered as a page or dashboard. Client-side JavaScript handles interface behavior. Server-side validation and configuration determine whether a particular feature is available.

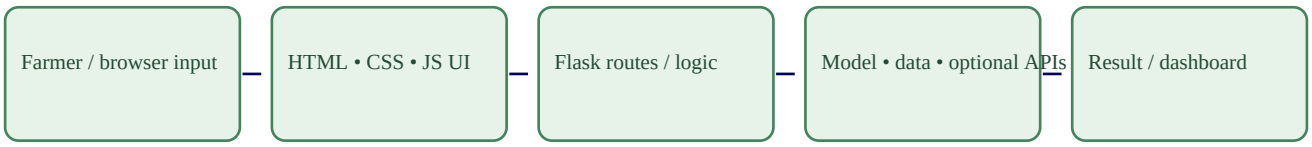

Figure 1. Typical request path. Integrations and model availability depend on configuration.

4. Feature guide

4.1 Crop recommendation

Purpose: help narrow crop choices using agricultural conditions such as soil and climate inputs. The user supplies supported values in the crop workflow; the backend/model layer processes the request; the interface presents a recommended crop or an error/availability message. README documents Random Forest/XGBoost as model families, but the exact active model version must be verified in the deployment. Inputs, units, dataset, and model metrics should be checked before relying on advice.

4.2 Fertilizer recommendation

Purpose: offer a fertilizer suggestion based on crop/soil or nutrient context. The feature is represented by the fertilizer recommendation module and interface. User inputs are submitted, validated, and used to produce an advisory result. Verify supported crops, feature schema, units, and model/data source in the active implementation; avoid treating an automated result as a prescription without local agronomic review.

4.3 Crop yield prediction

Purpose: estimate likely yield to support planning. A user provides the required crop, location/season, and environmental or farm inputs; the prediction component returns an estimate. The repository describes regression models. Estimates depend on training data, units, regional fit, and evaluation; no performance score is asserted in this guide.

4.4 Plant disease detection

Purpose: screen a plant image for visual patterns associated with disease. The user uploads a photo; image handling/sanitization and a CNN-based inference path may classify the image; the result is displayed for follow-up. Image quality and domain shift affect results. Treat output as screening, and seek expert confirmation for consequential decisions.

4.5 Soil classification and soil portal

Purpose: help users inspect soil-related information and related models. Soil inputs or images are processed by the corresponding module where connected. The repository includes soil classification model directories and a soil portal. The precise supported labels and model path should be confirmed against the deployed route and dataset.

4.6 Crop planning and crop calendar

Purpose: organize crop choices and seasonal tasks over time. Planning/calendar interfaces help users review timing and tasks. The benefit is turning recommendations into an actionable schedule; accuracy depends on region, crop variety, and local season configuration.

4.7 Weather and climate dashboards

Purpose: surface weather and climate context that informs farm decisions. Dashboard pages present available conditions or visual summaries. External feeds may require network/API configuration; indicate timestamps and source in use. Weather information is contextual and should not be read as guaranteed forecasts.

4.8 Irrigation and IoT monitoring

Purpose: expose monitoring/control workflows for water or field devices. Monitoring pages can display sensor-like readings, and irrigation control pages provide an interaction surface. Actual hardware actuation requires connected devices, credentials, network connectivity, and safety checks; a UI alone does not establish that physical control is active.

4.9 AI chatbot

Purpose: give users platform guidance and agriculture Q&A.; README describes a dual mode: Gemini-backed answers when configured and a JSON/rule-based fallback. The user sends a text prompt (and potentially an image through supported endpoints); the service returns a response. Region-specific advice requires location/context and validated sources. External AI availability is conditional.

4.10 Farmer community and forum

Purpose: enable peer discussion and knowledge exchange. Community/forum pages support posting and browsing conversations where configured. Moderation, authentication, and persistence depend on backend setup.

4.11 Marketplace, buyer and shopkeeper tools

Purpose: connect produce or agricultural supplies with market participants. Marketplace, buyer, shopkeeper, barter, procurement, and logistics pages provide listing, discovery, or coordination workflows. Transaction, inventory, and delivery status must be confirmed from actual configured services; UI listings do not themselves guarantee a completed sale.

4.12 Government schemes and financial support

Purpose: make scheme, loan, insurance, and finance information easier to find. These pages organize information and workflows. Eligibility and terms change; users should verify details with official sources and service providers.

4.13 Labour, equipment and farm operations

Purpose: help coordinate labor, equipment, and farm work. Labour alerts/portal and equipment pages provide discovery or management interfaces. Availability and booking depend on data freshness and actual service integration.

4.14 Sustainability and carbon features

Purpose: support awareness of sustainable practices and carbon-related tracking. Carbon and sustainability pages provide informational or recordkeeping workflows. Any quantified environmental impact needs a documented method, source data, and validation.

4.15 Accounts, profiles, notifications and administration

Purpose: provide user identity, preferences, saved information, and platform management. Registration/login and profile flows validate submitted details. The README notes name/username and Gmail-only registration rules enforced on client and server. Admin and audit pages are intended for operational oversight; exact role permissions must be verified in the running system.

4.16 Localization, voice and accessibility

Purpose: reduce interaction barriers across languages and input styles. Translation and voice-input scripts indicate support for localization and speech workflows. Actual language coverage, recognition quality, and accessibility conformance require testing with target users.

5. Technical implementation

Layer	Purpose	Repository evidence / notes
Frontend	Pages, layouts, forms, responsive styles, interaction logic	HTML files, CSS stylesheets, JavaScript modules; browser client
Web backend	Route requests, serve pages, coordinate services	app.py; Flask application; health endpoint described in README
ML inference	Transform supported inputs into predictions/classifications	TensorFlow/Keras, scikit-learn, OpenCV and model modules listed in requirements / README
Data and integrations	Store or retrieve application data; provide optional services	Firebase and external services are configuration-dependent; do not assume all are active
Operations	Run and package the app	Python environment, requirements.txt, Docker files and optional Node chatbot server

A normal local workflow documented in README is to create a Python virtual environment, install requirements, configure optional values in a private .env file, run app.py, and open the local application URL. The README identifies Flask as the main server and Node.js chatbot server as optional. Production deployment should use a WSGI server and deployment-specific secret management.

6. Cross-feature processing sequence

1. User opens a feature page and reads its instructions.

2. The page collects input and performs basic interface validation.

3. The request reaches the matching Flask route or configured service.

4. Server-side checks validate data, file type, and feature availability.

5. A model, database, or external service is invoked if configured.

6. The application formats a result, status, or actionable error.

7. The browser presents the response and the user chooses next steps.

7. Security, privacy and reliability

• Keep API keys and credentials in environment configuration; never commit .env values.

• Validate inputs in both client and server paths; do not trust browser-only checks.

• For uploaded images, restrict size/type and sanitize before processing.

• Use clear error messages without disclosing secrets or internal stack traces.

• Protect accounts and sensitive farm information with access controls appropriate to deployment.

• Check external service timeout, fallback, and stale-data behavior.

• Review model outputs for out-of-distribution inputs and communicate uncertainty.

8. Verification and evaluation

A meaningful evaluation should record feature, test data, preconditions, expected result, observed result, date, and environment. For machine-learning features, report dataset provenance, train/validation/test split, class balance, metrics, and limitations. For API integrations, test missing credentials, network failure, and rate limits. For UI, test desktop/mobile, keyboard navigation, validation, and error recovery. No test results or accuracy figures have been supplied, so this document deliberately makes no pass-rate claims.

Feature	Example verification
Crop/fertilizer advice	Valid input, missing values, boundary values, model unavailable
Yield estimate	Known test samples; metric calculation; unit consistency
Disease screening	Supported image, unsupported file, poor lighting, inference unavailable
Chatbot	Configured AI, fallback response, empty prompt, service outage
Accounts	Valid/invalid email, duplicate account, unauthorized route
Marketplace/community	Create/view/update flows, permissions, stale or empty data
IoT/control	No device, disconnected device, safe state, confirmed command outcome

9. Limits and responsible use

AgriTech is a decision-support platform. Model outputs can be wrong due to data quality, model coverage, environmental differences, and changing agricultural conditions. Not all pages imply a live external integration or physical device. Confirm recommendations with local agricultural experts; verify scheme, financial, weather, and market details with their authoritative providers.

10. Future improvements

- Publish a feature-to-route/model map with input schema and data lineage.
- Evaluate models on regionally representative datasets and disclose metrics.
- Add confidence and explanation cues with expert escalation.
- Improve low-bandwidth/offline workflows and regional language support.
- Integrate verified local data feeds with timestamps and source attribution.
- Test privacy, accessibility, security, and device-control safety.
- Capture field feedback and track outcomes without overstating impact.

11. Source map

This explanation was prepared from README.md and visible repository filenames/code surfaces, including app.py, crop recommendation, disease detection, yield prediction, soil portal, weather dashboard, irrigation, chatbot, community/forum, marketplace, account, and service pages. A code-level audit of each route, model artifact, and database schema would be needed to document exact endpoint names, algorithms, and production status. Treat README feature claims as intended capability until confirmed in the target deployment.

Repository setup references requirements.txt, .env.example, Dockerfile/docker-compose.yml, and server.js. Secrets from .env are intentionally excluded.